
Build GPT-2

Chen Ma

Min Zhu

Yuzhou Xu

Abstract

In this project, we fine-tuned GPT-2 on two tasks: sentiment analysis and sonnet generation. For sentiment analysis, we tested several training settings, including different numbers of unfrozen layers, embedding updates, and regularization. We also tried LoRA as a parameter-efficient method. For sonnet generation, we studied how training and decoding settings affect the final output. Our results show that both training choices and generation strategies can affect performance, especially on small datasets.

1 Introduction

In this project, we explored how GPT-2 can be adapted to two downstream tasks: sentiment analysis and sonnet generation. These two tasks are quite different, which makes them useful for studying how the same model behaves under different settings.

For sentiment analysis, we used the SST and CFIMDB datasets. Since both datasets are much smaller than the data used to pre-train GPT-2, we compared different fine-tuning choices to see what works best under limited data.

For sonnet generation, the model needs to generate text with both content and structure. This task is harder because the dataset is very small and the output must follow the sonnet format. We therefore tested different training settings and decoding methods to see how they affect the generated poems.

Overall, our goal is to understand how GPT-2 performs on small downstream tasks and which simple methods help it adapt better.

2 Related Work

This project investigates a downstream text classification task through Transfer Learning, demonstrating how large-scale Pre-trained Language Models (PLMs) can be adapted to domain-specific applications. Originally, Radford et al. (2019) demonstrated that language models optimized solely with unsupervised objectives on massive corpora exhibit robust zero-shot multitask capabilities [1].

When transferring these universal representations to target datasets like SST and CFIMDB, a foundational design choice emerges between static feature extraction and end-to-end parameter updates. As systematically investigated by Peters et al. (2019), full supervised fine-tuning generally achieves superior downstream performance compared to linear probing, particularly for tasks demanding intricate semantic coordination [2].

To balance optimization stability with performance capacity, Howard & Ruder (2018) pioneered the gradual unfreezing strategy, revealing a distinct feature hierarchy within deep networks where bottom layers preserve general syntax while top layers capture task-specific semantics [3]. However, expanding the optimization boundary—especially by unfreezing massive parameter blocks like token embeddings—frequently exposes high-capacity architectures to catastrophic forgetting and severe overfitting under low-resource constraints [4].

3 Data

We use two datasets for sentiment analysis: the Stanford Sentiment Treebank (SST) and CFIMDB. SST contains 11,855 short movie review snippets with five sentiment classes (from very negative to very positive), while CFIMDB consists of approximately 2,434 long movie reviews with binary

labels (negative or positive). No destructive preprocessing (e.g., stopword removal or punctuation stripping) is applied to preserve the linguistic structure learned during pre-training.

Another downstream task is sonnet generation, using The Sonnet Dataset, which consists of 154 sonnets written by Shakespeare. Each sonnet has 14 lines, following the rhyme scheme ABAB CDCD EFEF GG, where lines that are assigned the same letter ID rhyme with each other. This data set has three splits. The training set has 131 sonnets, the validation set has 12 sonnets, and the test set also has 12 sonnets. This data is intended to be used to train GPT-2 to generate new sonnets, based on the first 3 lines of the sonnets in the test set.

4 Methods

4.1 Downstream Task 1: Sentiment Analysis

First, we fine-tune the completed pre-trained model on two sentiment analysis datasets, SST and CFIMDB. The model with the default parameter settings in the provided code is used as the baseline. To systematically evaluate the impact of the amount, embedding configurations, and regularization in fine-tuning via hyperparameter search relative to the baseline, we first construct a controlled experimental matrix:

Table 1: Experimental Matrix

Group	Unfrozen Layers	Embedding	Regularization
A	0 (Linear Probing)	Frozen	None
B	1 (Last 1 Block)	Frozen	None
C	2 (Last 2 Blocks)	Frozen	None
D	-1 (Full Model)	Frozen	None
E	-1 (Full Model)	Unfrozen	None
F	-1 (Full Model)	Unfrozen	Dropout + WD
G	0 (Linear Probing)	Unfrozen	None
H	0 (Linear Probing)	Frozen	Dropout + WD

- **Amount:** The effect of progressively unfreezing more layers are compared among models $[A, B, C, D]$, which differ only in the number of trainable layers, ranging from linear probing to full fine-tuning.
- **Embedding:** We analyze the impact of updating embeddings by comparing $[D, E]$ under full-model fine-tuning and $[A, G]$ under linear probing.
- **Regularization:** We evaluate the effect of regularization by comparing $[E, F]$ (full-model setting) and $[A, H]$ (linear probing setting), focusing on dropout and weight decay.

4.2 Low-Rank Adaptation (LoRA)

In this section, we propose the use of Low-Rank Adaptation (LoRA) [5] to efficiently fine-tune a pre-trained GPT-2 model for downstream sentiment analysis tasks.

Given a pre-trained autoregressive language model (e.g., GPT-2) parameterized by Φ , standard full fine-tuning aims to maximize the log-likelihood of the conditional probability for a specific downstream task. However, full fine-tuning requires updating and storing a parameter gradient matrix $\Delta\Phi$ that is of the exact same dimension as the pre-trained model. As model scale increases, this approach becomes prohibitively expensive in terms of GPU memory and storage constraints.

4.2.1 Foundation of Low-Rank Adaptation (LoRA)

To overcome the computational and storage bottlenecks, we introduce LoRA. Assuming a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, we constrain its parameter updates ΔW by representing them in a low-rank subspace. Specifically, the weight update is factorized into the product of two low-rank matrices:

$$W = W_0 + \Delta W = W_0 + BA$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and the rank $r \ll \min(d, k)$. During training, W_0 is frozen and receives no gradient updates, while only A and B contain trainable parameters.

For an input $x \in \mathbb{R}^k$, the modified forward pass is computed as:

$$h = W_0x + \Delta Wx = W_0x + BAx$$

Initialization and Scaling: Following standard practice, we initialize matrix A using a random Gaussian distribution and matrix B with zeros, ensuring that $\Delta W = 0$ at the beginning of training. Furthermore, we scale ΔWx by a factor of $\frac{\alpha}{r}$, where α is a crucial hyperparameter tuned during experimentation. This scaling mechanism ensures that we do not need to retune the learning rate when varying the rank r .

4.2.2 Application to GPT-2 for Sentiment Classification

Early LoRA methods mainly targeted the query and value projection matrices in self-attention. In contrast, we apply LoRA to all major linear layers of the GPT-2 Transformer blocks, including the self-attention matrices (W_q, W_k, W_v), attention output projection ($W_{\text{att_dense}}$), and the MLP layers ($W_{\text{interm}}, W_{\text{out}}$), to better capture downstream semantic features.

For sequence classification, we use the hidden state of the last token, followed by a Dropout layer and a task-specific linear classification head to produce sentiment logits. During training, the GPT-2 backbone remains frozen, while only the LoRA matrices (A, B) and the classification head are optimized with AdamW, significantly reducing trainable parameters while maintaining strong performance.

4.3 Downstream Task 2: Sonnet Generation

We also fine-tuned the pre-trained model on the Sonnet Dataset to generate sonnets. The primary objective of this task is text generation: given the first 3 lines of a classical Shakespearean sonnet as a prompt, the model should ideally generate the remaining 11 lines while adhering to the specific stylistic, lexical, and structural constraints of the genre.

The hyperparameters in this downstream task contains two parts, parameters during training and parameters regard text generation. We designed a few sets of experiments to compare the performance of different learning rates and batchsizes, and impacts of different decoding strategies. We designed a experimental matrix of 10 evaluation runs, as shown in table 2

Table 2: Experimental Configuration Matrix for Sonnet Generation

Run Name	Learning Rate	Batch Size	Top- p	Repetition Penalty
Run 1 (No Penalty)	5×10^{-5}	8	0.90	1.00
Run 2 (Mild Penalty)	5×10^{-5}	8	0.90	1.10
Run 3 (Std Penalty)	5×10^{-5}	8	0.90	1.15
Run 4 (High Penalty)	5×10^{-5}	8	0.90	1.30
Run 5 (Conservative P)	5×10^{-5}	8	0.75	1.15
Run 6 (Creative P)	5×10^{-5}	8	0.95	1.15
Run 7 (Extreme P)	5×10^{-5}	8	0.99	1.15
Run 8 (LowLR SmallBS)	1×10^{-5}	4	0.90	1.15
Run 9 (MidLR SmallBS)	3×10^{-5}	4	0.90	1.15
Run 10 (MidLR LargeBS)	3×10^{-5}	8	0.90	1.15

4.3.1 Decoding and Regularization Strategies

To improve the quality and stability of sonnet generation, we explored several decoding and training strategies, including probabilistic decoding, early stopping, and repetition control.

Top- k and top- p decoding. To avoid repetitive sentences or random words, we combined top- k filtering and top- p sampling. First, top- k limits the model’s choices to the k most likely next words. Then, top- p dynamically filters those choices to only include words within the top p cumulative probability. This joint strategy effectively balances poetic creativity and grammatical correctness.

Early stopping based on validation loss. With a small dataset of only 131 sonnets, the model can easily overfit. To ensure it generalizes well, we monitored the validation loss using an Early Stopping mechanism with a patience of 3 epochs. If the validation loss fails to decrease for 3 rounds, training would stop automatically.

Repetition penalty with structural whitelisting. Standard repetition penalties lower the scores of recently used words to prevent repetitive looping. However, a uniform penalty ruins the poem’s layout by penalizing essential formatting tokens like newlines ($\backslash n$) and commas. To solve this, we created a punctuation and formatting whitelist $\mathcal{W}$:

$$\mathcal{W} = \{ '\backslash n', ',', ':', ';', '?', '!', '!', '!' \}$$

The penalty is only applied to tokens outside of $\mathcal{W}$. This adjustment maintains word diversity while protecting the poem’s line breaks and structural formatting.

5 Experiments

In this section, we first present outcome of the hyperparameter tuning and the advanced LoRA method for the sentiment analysis task, followed by the experimental results on the sonnet generation task.

5.1 Fine-Tuning and Hyperparameter Optimization

We first report the baseline results on the development sets. Last linear layer fine-tuning achieves 0.453 accuracy on SST and 0.867 on CFIMDB, while full-model fine-tuning improves performance to 0.513 and 0.980, respectively. Following the ablation framework in Section 4.1, we then conduct hyperparameter tuning for each configuration with Early Stopping to ensure fair comparison and reduce overfitting. The effects of the three components are analyzed below.

5.1.1 Amount

As shown in Table 3, performance improves as more transformer layers are unfrozen on both datasets. On SST, most gains are achieved by unfreezing only the top one or two layers, already approaching full fine-tuning. In contrast, CFIMDB shows larger improvements, with a 10% jump from unfreezing a single layer and further gains when all layers are unfrozen.

This difference likely stems from task complexity: SST involves short sentences, while CFIMDB contains longer and more complex reviews, requiring deeper adaptation. Overall, the optimal fine-tuning depth is dataset-dependent, balancing performance and computational cost.

5.1.2 Embedding

As demonstrated in Table 5, unfreezing the embedding layer exerts a detrimental impact on performance within the SST dataset. Conversely, it yields a beneficial effect on the CFIMDB dataset, manifested by a significant surge of approximately 7% in the linear probing configuration.

Table 3: Result of Amount Comparison

Group	UL	SST	CFIMDB
A	0	0.475	0.873
B	1	0.485	0.971
C	2	0.501	0.980
D	-1	0.526	0.984

Table 4: Result of Embedding Comparison

Group	UL	Embedding	SST	CFIMDB
A	0	Frozen	0.475	0.873
G	0	Frozen	0.456	0.943
D	-1	Unfrozen	0.526	0.984
E	-1	Unfrozen	0.522	0.984

5.1.3 Regularization

Mirroring the embedding adaptation trends, the impact of regularization (Dropout and Weight Decay) is highly dataset-specific as shown in Table 5. On SST, regularizers introduce a slight performance penalty across both low-capacity and high-capacity settings. Conversely, on CFIMDB, regularization exerts a good influence, delivering a 1.3% boost under linear probing while maintaining the optimal 0.984 plateau during full-model fine-tuning.

This shared divergence underscores that both embedding updates and regularization are heavily governed by sequence length: short, fine-grained sentences are easily over-constrained into minor underfitting, whereas dense, long-form reviews require weight smoothing to filter contextual noise and robustly anchor classification boundaries."

Table 5: Result of Regularization Comparison

Group	Unfrozen Layers	Regularization	SST	CFIMDB
A	0 (Linear Probing)	None	0.475	0.873
H	0 (Linear Probing)	Dropout+WD	0.472	0.886
E	-1 (Full Model)	None	0.522	0.984
F	-1 (Full Model)	Dropout+WD	0.520	0.984

5.2 Sentiment Analysis with LoRA

We evaluate LoRA-based parameter-efficient fine-tuning on the SST and CFIMDB. All experiments are conducted using GPT-2 small as the backbone model. We compare LoRA against the provided full

fine-tuning baseline from the CS224N starter implementation. For LoRA, we inject trainable low-rank matrices into the attention projection layers while freezing the original pretrained parameters.

In our experiments, we explore several LoRA hyperparameters by specifically varying the LoRA rank $r \in \{8, 16, 32\}$, scaling factor $\alpha \in \{16, 32, 64\}$, dropout rate $\in \{0.05, 0.1, 0.2, 0.3, 0.4\}$, learning rate $\in \{1e-5, 5e-5, 1e-4, 5e-4\}$, and batch size $\in \{8, 16, 32\}$. All models are trained using AdamW for 5–10 epochs depending on convergence behavior. For SST, we apply LoRA to both attention and feed-forward projection layers: `query`, `key`, `value`, `attention_dense`, `interm_dense`, `out_dense`. For CFIMDB, we find that restricting LoRA to only the attention projections: `query`, `value` achieves better generalization and training stability.

5.2.1 Main Results

Table 6 summarizes the performance of LoRA compared with the baseline fine-tuning methods.

Table 6: Comparison between full fine-tuning and LoRA-based adaptation.

Method	Trainable Params (%)	SST Acc	CFIMDB Acc
Last Linear Layer	~ 0.1	0.462	0.861
Full Fine-Tuning	100.0	0.513	0.976
LoRA (Ours)	2.08	0.532	0.984

Our LoRA implementation outperforms the provided full fine-tuning baseline on both datasets while updating only a small fraction of the model parameters. On SST, LoRA achieves an accuracy of 0.532 while training only 2.08% of the total parameters. On CFIMDB, LoRA reaches 0.984 development accuracy with less than 1% trainable parameters.

These results suggest that low-rank adaptation is sufficient for downstream sentiment classification tasks and can match or exceed full fine-tuning performance with significantly lower training cost.

5.2.2 Analysis Results

We observe that very small learning rates ($1e-5$) lead to underfitting, while large learning rates ($5e-4$) cause unstable training. Moderate dropout values (0.1–0.2) provide the best generalization, whereas excessive dropout weakens adaptation under low-rank parameterization.

Figure 1 shows the training dynamics of the best SST configuration.

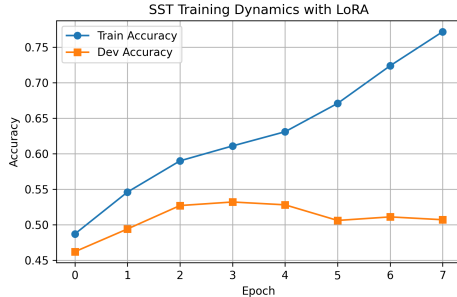


Figure 1: Training and development accuracy curves on SST using LoRA fine-tuning.

SST begins to overfit after 3–4 epochs, while CFIMDB shows more stable optimization and better generalization. We attribute this difference to SST being a more difficult 5-way classification task, whereas CFIMDB is a binary sentiment classification task with stronger sentiment signals.

LoRA also demonstrates strong parameter efficiency. The best SST and CFIMDB models train only 2.08% and 0.94% of total parameters, respectively, while still outperforming full fine-tuning. These results show that parameter-efficient fine-tuning can effectively adapt pretrained language models with substantially lower memory and optimization costs.

5.3 Sonnet Generation Task

The sonnet generation task contains two parts, training and decoding. The quality of the generated sonnets not only depend on the training process, but also highly on the decoding process.

5.3.1 Quantitative Results

After completing the full hyperparameter search matrix, the quantitative evaluation metrics for all configurations are documented in Table 7. The metrics include the final validation cross-entropy loss and the chrF score evaluated on the development set.

Table 7: Quantitative Evaluation Results of Sonnet Generation Models

Configuration	Val Loss	chrF Score
Run 1 (No Penalty)	3.8039	27.02
Run 2 (Mild Penalty)	3.8039	29.51
Run 3 (Std Penalty)	3.8039	30.10
Run 4 (High Penalty)	3.8039	29.94
Run 5 (Conservative P)	3.8039	29.87
Run 6 (Creative P)	3.8039	29.54
Run 7 (Extreme P)	3.8039	30.25
Run 8 (LowLR SmallBS)	3.8705	29.99
Run 9 (MidLR SmallBS)	3.8761	29.91
Run 10 (MidLR LargeBS)	3.8115	30.51

A finding in Table 7 is that **Run 7 (Extreme P, $p = 0.99$)** achieves the highest objective alignment score (chrF = 30.25) among all decoding ablation variants, seemingly contradicting the convention that near-maximal p values introduce catastrophic noise. This is because before top- p is applied, the potential vocabulary has already been filtered by the top- k process. Thus, top- p is just functional on the top- k words, resulting in unusual phenomenon.

5.3.2 Qualitative Analysis and Degenerative Looping

Although chrF gives a general score, manual inspection shows that decoding strategy has a clear effect on generation quality.

In **Run 1**, we removed the repetition penalty ($\rho = 1.0$). The model then often repeated the same short phrases and produced degenerate loops, even though the validation loss was still low.

In **Run 3**, we used a moderate repetition penalty ($\rho = 1.15$). This reduced repetition and made the generated text more coherent.

The side-by-side comparison below shows the difference between the two settings. The left column is the output without repetition penalty, and the right column is the output with repetition penalty.

Table 8: Qualitative comparison between decoding without and with repetition penalty.

Run 1: No Repetition Penalty ($\rho = 1.0$)	Run 3: Standard Penalty ($\rho = 1.15$)
...	...
<i>Thy tongue, my tongue, my tongue,</i>	<i>When in my presence make thee mine,</i>
<i>O, that thou art my heart's scribe;</i>	<i>And when thou art gone, what dost desire</i>
<i>Thy lips, my lips, my lips,</i>	<i>To see the same woman again?</i>
<i>Thy lips, my lips, my lips,</i>	<i>My friend, all is well; but that which will not do,</i>
...	...

As shown in Table 8, removing the repetition penalty leads to repeated phrases and weaker semantic flow. With a proper penalty, the output becomes more diverse and better follows the sonnet structure. This shows that repetition control is important for our generation task.

6 Conclusion

In this project, we fine-tuned GPT-2 on sentiment analysis and sonnet generation. For sentiment analysis, we found that unfreezing more layers consistently boosts performance, while embedding updates and regularization exhibit highly dataset-specific effects, and that LoRA can be a useful way to train the model efficiently. For sonnet generation, we found that decoding settings such as repetition penalty and top- p sampling have a large effect on the final output.

Overall, our experiments show that on small datasets, simple training and decoding choices can matter a lot. Even when the final score is not very high, the process still shows how GPT-2 can be adapted to different tasks and how sensitive it is to design choices.

References

- [1] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [2] Matthew E Peters, Sebastian Ruder, and Noah A Smith. To tune or not to tune? adapting pretrained representations to diverse tasks. In *Proceedings of the 4th Workshop on Representation Learning for NLP (RepL4NLP-2019)*, pages 7–14, 2019.
- [3] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 328–339, 2018.
- [4] Cheolhyoung Lee, Kyunghyun Cho, and Wanmo Kang. Mixout: Effective regularization to finetune large-scale pretrained language models. *arXiv preprint arXiv:1909.11299*, 2019.
- [5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021.

References follow the acknowledgments in the camera-ready paper. Use unnumbered first-level heading for the references. Any choice of citation style is acceptable as long as you are consistent. It is permissible to reduce the font size to `small` (9 point) when listing the references. Note that the Reference section does not count towards the page limit.

A Code Available

All codes of our work can be found in github.